

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 14 – SORTING



NAMA: Dinar Azzahra Putri Setiadi

NIM: 109082500176

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

Sorting atau pengurutan data dalam Algoritma dan Pemrograman (Alpro) merupakan proses mengatur sekelompok data ke dalam urutan tertentu berdasarkan aturan atau kunci (*key*) tertentu. Secara umum, terdapat dua jenis urutan utama yang digunakan, yaitu *ascending* (mengurutkan data dari nilai terkecil ke terbesar) dan *descending* (mengurutkan data dari nilai terbesar ke terkecil). Proses pengurutan ini sangat krusial dalam dunia ilmu komputer karena struktur data yang terurut akan meningkatkan efisiensi dan kecepatan algoritma lain secara signifikan, terutama pada operasi pencarian (*searching*) dan pencocokan data.

Di dalam praktiknya, terdapat berbagai macam metode sorting yang masing-masing memiliki karakteristik, kompleksitas waktu, dan penggunaan memori yang berbeda-beda. Metode yang umum dipelajari di antaranya adalah *Bubble Sort*, *Selection Sort*, dan *Insertion Sort* untuk kategori algoritma sederhana, serta *Merge Sort* dan *Quick Sort* yang menggunakan pendekatan *divide and conquer* untuk menangani data skala besar secara lebih efisien. Pemilihan algoritma pengurutan yang tepat sangat bergantung pada volume data, kondisi awal data (apakah sudah hampir terurut atau acak total), serta keterbatasan memori pada sistem yang digunakan.

B. Guided

1. [Sel Sort Array]

```
2. package main
3.
4. import "fmt"
5.
6. func selecetionSortArray(angka *[5]int){
7.     var idx_min, i, j int
8.     for i = 0; i < len(angka) - 1; i++ { //loop luar (iterasi
        sortingnya)
9.         idx_min = i
10.        for j = i + 1; j < len(angka); j++ { //loop dalam (mencari
            nilai ekstrim)
11.            if angka[j] <= angka[idx_min] { //sorting terkecil ke
                terbesar
12.                idx_min = j
13.            }
14.        }
15.        if idx_min != i {
16.            angka[i], angka[idx_min] = angka[idx_min], angka[i]
17.        }
18.    }
19.}
20.
21. func main(){
22.     var arrAngka [5]int
23.
24.     for i := 0; i < len(arrAngka); i++ {
25.         fmt.Printf("Masukkan data angka ke-%d : ", i)
26.         fmt.Scan(&arrAngka[i])
27.     }
28.     fmt.Println()
29.
30.     fmt.Println("=== SEBELUM DISORTING ===")
31.     for i := 0; i < len(arrAngka); i++ {
32.         fmt.Print(arrAngka[i], " - ")
33.     }
34.
35.     fmt.Println("\n=== SETELAH DISORITNG ===")
36.     selecetionSortArray(&arrAngka)
37.     for i := 0; i < len(arrAngka); i++ {
38.         fmt.Print(arrAngka[i], " - ")
39.     }
```

Output :

```
PROBLEMS 2 OUTPUT TERMINAL DEBUG CONSOLE PORTS
PS D:\109082500176_Dinar-Azzahra-Putri-Setiadi\Week 12 - Modul 14\Guided\Guided-1> go run SelSortArray.go
Masukkan data angka ke-0 : 10
Masukkan data angka ke-1 : 15
Masukkan data angka ke-2 : 17
Masukkan data angka ke-3 : 19
Masukkan data angka ke-4 : 21

=== SEBELUM DISORTING ===
10 - 15 - 17 - 19 - 21 -
=== SETELAH DISORTING ===
10 - 15 - 17 - 19 - 21 -
```

2. [Sel Sort Struct]

```
3. package main
4.
5. import "fmt"
6.
7. type mahasiswa struct {
8.     nama, nim string
9.     IPK      float64
10.}
11.
12.func SelectionSortArray(sMHS *[5]mahasiswa) {
13.    var idx_min, i, j int
14.    for i = 0; i < len(sMHS)-1; i++ { //loop luar
15.        idx_min = i
16.        for j = i + 1; j < len(sMHS); j++ { //loop dalam mencari
            nilai ekstrim
17.            if sMHS[j].IPK < sMHS[idx_min].IPK { //sorting terkecil
                ke terbesar
18.                idx_min = j
19.            }
20.        }
21.        if idx_min != i {
22.            sMHS[i], sMHS[idx_min] = sMHS[idx_min], sMHS[i]
23.        }
24.    }
25.}
26.
27.func main() {
28.    var structMHS [5]mahasiswa
29.
```

```

30.     for i := 0; i < len(structMHS); i++ {
31.         fmt.Printf("--- Masukkan Data Mahasiswa Ke-%d ---\n", i)
32.         fmt.Print("Nama : ")
33.         fmt.Scan(&structMHS[i].nama)
34.         fmt.Print("NIM : ")
35.         fmt.Scan(&structMHS[i].nim)
36.         fmt.Print("IPK : ")
37.         fmt.Scan(&structMHS[i].IPK)
38.     }
39.     fmt.Println()
40.
41.     fmt.Println(" === SEBELUM SORITNG === ")
42.     for i := 0; i < len(structMHS); i++ {
43.         fmt.Printf("--- Data Mahasiswa Ke-%d ---\n", i)
44.         fmt.Printf("Nama : %s\n", structMHS[i].nama)
45.         fmt.Printf("NIM : %s\n", structMHS[i].nim)
46.         fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
47.     }
48.     fmt.Println("=====")
49.     fmt.Println()
50.
51.     SelectionSortArray(&structMHS)
52.     fmt.Println(" === SETELAH SORITNG === ")
53.     for i := 0; i < len(structMHS); i++ {
54.         fmt.Printf("--- Data Mahasiswa Ke-%d ---\n", i)
55.         fmt.Printf("Nama : %s\n", structMHS[i].nama)
56.         fmt.Printf("NIM : %s\n", structMHS[i].nim)
57.         fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
58.     }
59.     fmt.Println("=====")
60.     fmt.Println()
61. }

```

Output :

```
PROBLEMS 3 OUTPUT TERMINAL DEBUG CONSOLE PORTS
PS D:\109082500176_Dinar-Azzahra-Putri-Setiadi\Week 12 - Modul 14\Guided\Guided-2> go run SelSortStruct.go
--- Masukkan Data Mahasiswa Ke-0 ---
Nama : key
NIM : 234
IPK : 3.00
--- Masukkan Data Mahasiswa Ke-1 ---
Nama : mia
NIM : 145
IPK : 3.50
--- Masukkan Data Mahasiswa Ke-2 ---
Nama : shea
NIM : 657
IPK : 3.90
--- Masukkan Data Mahasiswa Ke-3 ---
Nama : noya
NIM : 356
IPK : 4.00
--- Masukkan Data Mahasiswa Ke-4 ---
Nama : abel
NIM : 892
IPK : 4.00

=== SEBELUM SORITNG ===
--- Data Mahasiswa Ke-0 ---
Nama : key
NIM : 234
IPK : 3.00
--- Data Mahasiswa Ke-1 ---
Nama : mia
NIM : 145
IPK : 3.50
```

```
PROBLEMS 3 OUTPUT TERMINAL DEBUG CONSOLE PORTS

--- Data Mahasiswa Ke-2 ---
Nama : shea
NIM : 657
IPK : 3.90
--- Data Mahasiswa Ke-3 ---
Nama : noya
NIM : 356
IPK : 4.00
--- Data Mahasiswa Ke-4 ---
Nama : abel
NIM : 892
IPK : 4.00
=====

=== SETELAH SORITNG ===
--- Data Mahasiswa Ke-0 ---
Nama : key
NIM : 234
IPK : 3.00
--- Data Mahasiswa Ke-1 ---
Nama : mia
NIM : 145
IPK : 3.50
--- Data Mahasiswa Ke-2 ---
Nama : shea
NIM : 657
IPK : 3.90
--- Data Mahasiswa Ke-3 ---
Nama : noya
NIM : 356
IPK : 4.00
--- Data Mahasiswa Ke-4 ---
Nama : abel
NIM : 892
IPK : 4.00
=====
```

3. [Insertion Sort String]

```
4. package main
5.
6. import "fmt"
7.
8. const nMax int = 100
9.
10. type arrString [nMax]string
11.
12. func insertionSortString(T *arrString, n int) {
13.     var temp string
14.     var i, j int
15.
16.     i = 1
17.     for i <= n-1 {
18.         j = i
19.         temp = T[j]
20.
21.         for j > 0 && temp < T[j-1] {
22.             T[j] = T[j-1]
23.             j = j - 1
24.         }
25.
26.         T[j] = temp
27.         i = i + 1
28.     }
29. }
30.
31. func main() {
32.     var data arrString
33.     var n int
34.
35.     fmt.Scan(&n)
36.
37.     for i := 0; i < n; i++ {
38.         fmt.Scan(&data[i])
39.     }
40.
41.     insertionSortString(&data, n)
42.
43.     for i := 0; i < n; i++ {
44.         fmt.Print(data[i], " ")
45.     }
```

```
46. }
```

Output :

```
PROBLEMS 4 OUTPUT TERMINAL DEBUG CONSOLE PORTS
PS D:\109082500176_Dinar-Azzahra-Putri-Setiadi\Week 12 - Modul 14\Guided\Guided-3> go run insertionSortString.go
11 12 13 14 15
12 14 18 20 22
13 15 17 19 21
12 12 13 13 14 14 15 15 18 20 22
PS D:\109082500176_Dinar-Azzahra-Putri-Setiadi\Week 12 - Modul 14\Guided\Guided-3>
```

4. [Insertion Sort]

```
5. package main
6.
7. import "fmt"
8.
9. const nMax int = 4321
10.
11. type arrInt [nMax]int
12.
13. func insertionSort(T *arrInt, n int) {
14.     /* I.S. terdefinisi array T yang berisi n bilangan bulat
15.        F.S. array T terurut secara mengecil atau descending
16.        dengan INSERTION SORT */
17.     var temp, i, j int
18.
19.     i = 1
20.     for i <= n-1 {
21.         j = i
22.         temp = T[j]
23.
24.         for j > 0 && temp > T[j-1] {
25.             T[j] = T[j-1]
26.             j = j - 1
27.         }
28.
29.         T[j] = temp
30.         i = i + 1
31.     }
32. }
33.
```

```

34. func main() {
35.     var data arrInt
36.     var n int
37.
38.     fmt.Scan(&n)
39.
40.     for i := 0; i < n; i++ {
41.         fmt.Scan(&data[i])
42.     }
43.
44.     insertionSort(&data, n)
45.
46.     for i := 0; i < n; i++ {
47.         fmt.Print(data[i], " ")
48.     }
49. }

```

Output :

```

PROBLEMS 2 OUTPUT TERMINAL DEBUG CONSOLE PORTS
PS D:\109082500176_Dinar-Azzahra-Putri-Setiadi\Week 12 - Modul 14\Guided\Guided-4> go run insertionSort.go
13 15 17 19 21
12 14 18 20 22
23 25 27 29 31
29 27 25 23 22 21 20 19 18 17 15 14 12

```

C. Unguided

1. Soal Unguided 1 (Insertion Sort)

Buatlah sebuah program yang digunakan untuk membaca data integer seperti contoh yang diberikan di bawah ini, kemudian diurutkan (menggunakan metoda *insertion sort*), dan memeriksa apakah data yang terurut berjarak sama terhadap data sebelumnya. Masukan terdiri dari sekumpulan bilangan bulat yang diakhiri oleh bilangan negatif. Hanya bilangan non negatif saja yang disimpan ke dalam array. Keluaran terdiri dari dua baris. Baris pertama adalah isi dari array setelah dilakukan pengurutan, sedangkan baris kedua adalah status jarak setiap bilangan yang ada di dalam array. "Data berjarak x" atau "data berjarak tidak tetap".

Jawaban :

```

package main

import (
    "fmt"
)

```

```

func main() {
    var input int
    var data []int

    for {
        fmt.Scan(&input)
        if input < 0 {
            break
        }
        data = append(data, input)
    }

    if len(data) == 0 {
        return
    }

    for i := 1; i < len(data); i++ {
        key := data[i]
        j := i - 1
        for j >= 0 && data[j] > key {
            data[j+1] = data[j]
            j = j - 1
        }
        data[j+1] = key
    }

    for i, val := range data {
        fmt.Printf("%d", val)
        if i < len(data)-1 {
            fmt.Print(" ")
        }
    }
    fmt.Println()

    isConstant := true
    var diff int

    if len(data) > 1 {
        diff = data[1] - data[0]
        for i := 1; i < len(data)-1; i++ {
            if data[i+1]-data[i] != diff {
                isConstant = false
                break
            }
        }
    } else {
        diff = 0
    }

    if isConstant {
        fmt.Printf("Data berjarak %d\n", diff)
    }
}

```

```

    } else {
        fmt.Println("Data berjarak tidak tetap")
    }
}

```

Output :



```

PROBLEMS 2 OUTPUT TERMINAL DEBUG CONSOLE PORTS
PS D:\109082500176_Dinar-Azzahra-Putri-Setiadi\Week 12 - Modul 14\Unguided\Unguided-1> go run angka.go
31 13 25 43 1 7 19 37 -5
1 7 13 19 25 31 37 43
Data berjarak 6
PS D:\109082500176_Dinar-Azzahra-Putri-Setiadi\Week 12 - Modul 14\Unguided\Unguided-1> go run angka.go
4 40 14 8 26 1 38 2 32 -31
1 2 4 8 14 26 32 38 40
Data berjarak tidak tetap

```

Penjelasan :

Program ini bekerja dengan membaca input angka secara berulang dan memasukkannya ke dalam *array*, lalu perulangan akan langsung berhenti ketika mendeteksi angka negatif. Setelah semua angka non-negatif tersimpan, program mengurutkannya secara mendaki (*ascending*) menggunakan algoritma Insertion Sort, yang bekerja dengan cara menyisipkan setiap elemen ke posisi yang tepat di sebelah kirinya. Setelah data terurut dicetak pada baris pertama, program mengukur selisih antara elemen pertama dan kedua sebagai acuan awal. Program kemudian memeriksa seluruh data; jika semua elemen yang bersebelahan memiliki selisih yang konsisten sama, program akan mencetak status "Data berjarak x", namun jika ditemukan ada selisih yang berbeda, program akan mencetak status "Data berjarak tidak tetap".

2. Soal Unguided 2 (Insertion Sort)

Sebuah program perpustakaan digunakan untuk mengelola data buku di dalam suatu Perpustakaan. Masukan terdiri dari beberapa baris. Baris pertama adalah bilangan bulat *N* yang menyatakan banyaknya data buku yang ada di dalam perpustakaan. *N* baris berikutnya, masing-masingnya adalah data buku sesuai dengan atribut atau field pada struct. Baris terakhir adalah bilangan bulat yang menyatakan rating buku yang akan dicari. Keluaran terdiri dari beberapa baris. Baris pertama adalah data buku terfavorit, baris kedua adalah lima judul buku dengan rating tertinggi, selanjutnya baris terakhir adalah data buku yang dicari sesuai rating yang diberikan pada masukan baris terakhir.

Jawaban :

```

package main

import (
    "fmt"
)

const nMax = 7919

type Buku struct {
    id, judul, penulis, penerbit string
    eksemplar, tahun, rating      int
}

type DaftarBuku [nMax]Buku

func DaftarkanBuku(pustaka *DaftarBuku, n *int) {
    fmt.Scan(n)
    for i := 0; i < *n; i++ {
        fmt.Scan(&pustaka[i].id, &pustaka[i].judul,
            &pustaka[i].penulis, &pustaka[i].penerbit,
            &pustaka[i].eksemplar, &pustaka[i].tahun,
            &pustaka[i].rating)
    }
}

func CetakTerfavorit(pustaka DaftarBuku, n int) {
    if n <= 0 {
        return
    }
    terfavorit := pustaka[0]
    for i := 1; i < n; i++ {
        if pustaka[i].rating > terfavorit.rating {
            terfavorit = pustaka[i]
        }
    }
    fmt.Printf("%s %s %s %d\n", terfavorit.judul,
        terfavorit.penulis, terfavorit.penerbit,
        terfavorit.tahun)
}

func UrutBuku(pustaka *DaftarBuku, n int) {
    for i := 1; i < n; i++ {
        key := pustaka[i]
        j := i - 1
        for j >= 0 && pustaka[j].rating < key.rating {
            pustaka[j+1] = pustaka[j]
            j = j - 1
        }
        pustaka[j+1] = key
    }
}

```

```

func Cetak5Terbaru(pustaka DaftarBuku, n int) {
    limit := 5
    if n < 5 {
        limit = n
    }
    for i := 0; i < limit; i++ {
        fmt.Print(pustaka[i].judul)
        if i < limit-1 {
            fmt.Print(" ")
        }
    }
    fmt.Println()
}

func CariBuku(pustaka DaftarBuku, n int, r int) {
    low := 0
    high := n - 1
    foundIdx := -1

    for low <= high {
        mid := (low + high) / 2
        if pustaka[mid].rating == r {
            foundIdx = mid
            break
        } else if pustaka[mid].rating < r {
            high = mid - 1
        } else {
            low = mid + 1
        }
    }

    if foundIdx != -1 {
        b := pustaka[foundIdx]
        fmt.Printf("%s %s %s %d %d %d\n", b.judul,
b.penulis, b.penerbit, b.tahun, b.eksemplar, b.rating)
    } else {
        fmt.Println("Tidak ada buku dengan rating
seperti itu")
    }
}

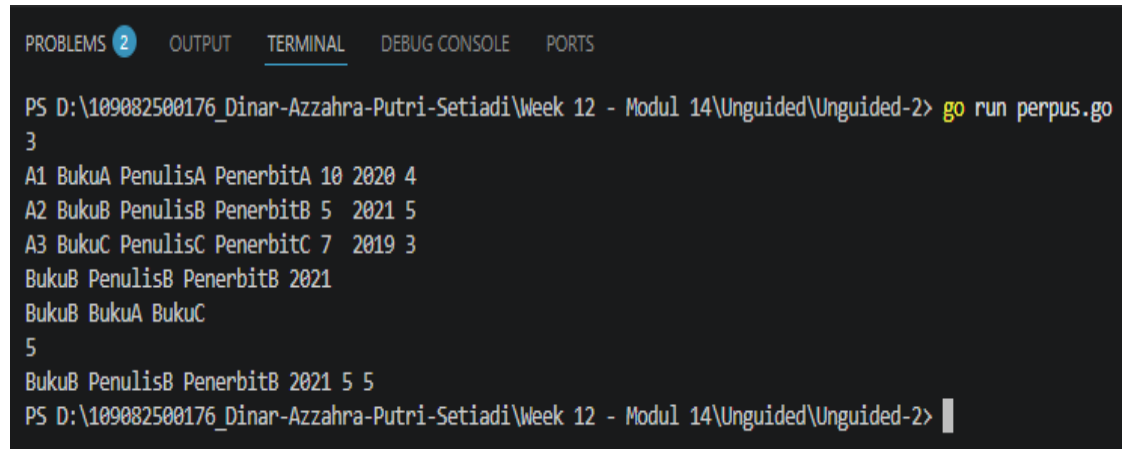
func main() {
    var pustaka DaftarBuku
    var n, ratingCari int

    DaftarkanBuku(&pustaka, &n)
    CetakTerfavorit(pustaka, n)
    UrutBuku(&pustaka, n)
    Cetak5Terbaru(pustaka, n)
}

```

```
        fmt.Scan(&ratingCari)
        CariBuku(pustaka, n, ratingCari)
    }
```

Output :



```
PROBLEMS 2 OUTPUT TERMINAL DEBUG CONSOLE PORTS
PS D:\109082500176_Dinar-Azzahra-Putri-Setiadi\Week 12 - Modul 14\Unguided\Unguided-2> go run perpus.go
3
A1 BukuA PenulisA PenerbitA 10 2020 4
A2 BukuB PenulisB PenerbitB 5 2021 5
A3 BukuC PenulisC PenerbitC 7 2019 3
BukuB PenulisB PenerbitB 2021
BukuB BukuA BukuC
5
BukuB PenulisB PenerbitB 2021 5 5
PS D:\109082500176_Dinar-Azzahra-Putri-Setiadi\Week 12 - Modul 14\Unguided\Unguided-2> |
```

Penjelasan :

Program ini dirancang modular untuk mengelola sekumpulan data buku perpustakaan menggunakan struktur struct dan array berukuran statis. Pertama, prosedur DaftarkanBuku mengisi data buku secara sekuensial dari input pengguna sebanyak jumlah N. Selanjutnya, program mencari dan menampilkan identitas buku terfavorit melalui fungsi pencarian nilai maksimum linear pada prosedur CetakTerfavorit sebelum data diubah. Setelah itu, fungsi UrutBuku menerapkan algoritma *Insertion Sort* untuk menyusun data buku secara menurun (*descending*) berdasarkan ratingnya, yang memudahkan prosedur Cetak5Terbaru untuk langsung mengambil dan menampilkan maksimal lima judul buku teratas dari barisan terdepan array. Di tahap akhir, program membaca kriteria rating yang dicari lalu mengeksekusi prosedur CariBuku dengan metode pencarian biner (*Binary Search*) untuk melacak data buku secara cepat dan efisien pada array terurut tersebut.

3. Soal Unguided 3 (Selection Sort)

Hercules, preman terkenal seantero ibukota, memiliki kerabat di banyak daerah. Tentunya Hercules sangat suka mengunjungi semua kerabatnya itu. Diberikan masukan nomor rumah dari semua kerabatnya di suatu daerah, buatlah program rumah kerabat yang akan menyusun nomor-nomor rumah kerabatnya secara terurut membesar menggunakan algoritma selection sort. Masukan dimulai dengan sebuah integer n ($0 < n < 1000$), banyaknya daerah kerabat Hercules tinggal. Isi n baris berikutnya selalu dimulai dengan sebuah integer m ($0 < m < 1000000$) yang menyatakan banyaknya rumah kerabat di daerah tersebut, diikuti dengan rangkaian

bilangan bulat positif, nomor rumah para kerabat. Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurutmembesar dimasing-masing daerah.

Jawaban :

```
package main

import (
    "fmt"
)

func main() {
    var n, m int
    if fmt.Scan(&n); n <= 0 {
        return
    }

    for i := 0; i < n; i++ {
        fmt.Scan(&m)

        rumah := make([]int, m)
        for j := 0; j < m; j++ {
            fmt.Scan(&rumah[j])
        }

        for step := 0; step < m-1; step++ {
            minIdx := step
            for k := step + 1; k < m; k++ {
                if rumah[k] < rumah[minIdx] {
                    minIdx = k
                }
            }

            rumah[step], rumah[minIdx] = rumah[minIdx],
rumah[step]
        }

        for idx, val := range rumah {
            fmt.Printf("%d", val)
            if idx < m-1 {
                fmt.Print(" ")
            }
        }
        fmt.Println()
    }
}
```

Output :

```
PROBLEMS 2 OUTPUT TERMINAL DEBUG CONSOLE PORTS
PS D:\109082500176_Dinar-Azzahra-Putri-Setiadi\Week 12 - Modul 14\Unguided\Unguided-4> go run kerabatdekat.go
3
5 2 1 7 9 13
1 7 9 13 2
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 9 4
PS D:\109082500176_Dinar-Azzahra-Putri-Setiadi\Week 12 - Modul 14\Unguided\Unguided-4> |
```

Penjelasan :

Program bernama rumahkerabat ini dibuat untuk mengurutkan nomor rumah kerabat Hercules secara membesar (*ascending*) di setiap daerah yang dikunjunginya. Pada awalnya, program membaca nilai integer n sebagai jumlah total daerah, lalu masuk ke dalam perulangan sebanyak n kali untuk memproses setiap daerah secara bergantian. Di dalam perulangan tersebut, program membaca jumlah rumah (m) pada daerah yang sedang aktif, mengalokasikan sebuah *slice* dinamis seukuran m, dan mengisinya dengan rangkaian nomor rumah yang diinputkan. Proses pengurutan data kemudian dieksekusi menggunakan algoritma Selection Sort, yang bekerja dengan cara memindai array untuk mencari elemen dengan nilai terkecil, lalu menukarkan posisinya ke urutan paling depan yang belum terurut secara bertahap hingga seluruh data selesai diproses. Setelah nomor rumah di suatu daerah berhasil diurutkan, program langsung mencetaknya dalam satu baris dengan pemisah spasi, kemudian berpindah ke baris baru untuk mengulangi prosedur yang sama pada daerah berikutnya.

4. Soal Unguided 3 (Selection Sort)

Belakangan diketahui ternyata Hercules itu tidak berani menyeberang jalan, maka selalu diusahakan agar hanya menyeberang jalan sesedikit mungkin, hanya diujung jalan. Karena nomor rumah sisi kiri jalan selalu ganjil dan sisi kanan jalan selalu genap, maka buatlah program kerabat dekat yang akan menampilkan nomor rumah mulai dari nomor yang ganjil lebih dulu terurut membesar dan kemudian menampilkan nomor rumah dengan nomor genap terurut mengecil. Format Masukan masih persis sama seperti sebelumnya. Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar untuk nomor ganjil, diikuti dengan terurut mengecil untuk nomor genap, dimasing-masing daerah.

Jawaban :

```
package main
```

```

import (
    "fmt"
)

func main() {
    var n, m, nomor int

    if fmt.Scan(&n); n <= 0 {
        return
    }

    for i := 0; i < n; i++ {
        fmt.Scan(&m)

        var ganjil []int
        var genap []int

        for j := 0; j < m; j++ {
            fmt.Scan(&nomor)
            if nomor%2 != 0 {
                ganjil = append(ganjil, nomor)
            } else {
                genap = append(genap, nomor)
            }
        }

        for step := 0; step < len(ganjil)-1; step++ {
            minIdx := step
            for k := step + 1; k < len(ganjil); k++ {
                if ganjil[k] < ganjil[minIdx] {
                    minIdx = k
                }
            }
            ganjil[step], ganjil[minIdx] =
ganjil[minIdx], ganjil[step]
        }

        for step := 0; step < len(genap)-1; step++ {
            maxIdx := step
            for k := step + 1; k < len(genap); k++ {
                if genap[k] > genap[maxIdx] {
                    maxIdx = k
                }
            }
            genap[step], genap[maxIdx] = genap[maxIdx],
genap[step]
        }

        var hasil []int
        hasil = append(hasil, ganjil...)
    }
}

```

```

        hasil = append(hasil, genap...)

        for idx, val := range hasil {
            fmt.Printf("%d", val)
            if idx < len(hasil)-1 {
                fmt.Print(" ")
            }
        }
        fmt.Println()
    }
}

```

Output :

```

PS D:\ALPRO 2\109082500176_Dinar-Azzahra-Putri-Setiadi\Week 11 - Modul 12\Unguided\Unguided-3> go run posisidata.go
12 534
1 3 8 16 32 123 323 323 534 543 823 999
8
PS D:\ALPRO 2\109082500176_Dinar-Azzahra-Putri-Setiadi\Week 11 - Modul 12\Unguided\Unguided-3> go run posisidata.go
12 535
1 3 8 16 32 123 323 323 534 543 823 999
TIDAK ADA
PS D:\ALPRO 2\109082500176_Dinar-Azzahra-Putri-Setiadi\Week 11 - Modul 12\Unguided\Unguided-3>

```

Penjelasan :

Program bernama kerabat dekat ini dirancang untuk memisahkan sekaligus mengurutkan nomor rumah berdasarkan jenis bilangan ganjil dan genap di setiap daerah. Saat membaca data masukan di setiap daerah, program langsung memilah nomor rumah; nomor ganjil dimasukkan ke dalam *slice* ganjil, sementara nomor genap dialihkan ke dalam *slice* genap. Selanjutnya, program menerapkan algoritma Selection Sort sebanyak dua kali dengan aturan yang berbeda: pada kelompok ganjil dilakukan pengurutan membesar (*ascending*) dengan mencari nilai terkecil, sedangkan pada kelompok genap dilakukan pengurutan mengecil (*descending*) dengan mencari nilai terbesar. Di tahap akhir, kedua kelompok data yang telah teratur tersebut digabungkan kembali secara berurutan—ganjil terlebih dahulu, disusul genap lalu dicetak ke layar dalam satu baris sebelum program melanjutkan pemrosesan ke daerah berikutnya.

D. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, berfokus pada pemahaman dan implementasi struktur data lanjut serta algoritma pengurutan (*sorting*) dan pencarian (*searching*) dasar yang diimplementasikan secara modular. Melalui studi kasus yang diberikan, mahasiswa diajarkan bagaimana mengelola sekumpulan data bertipe homogen menggunakan array statis maupun dinamis (*slice*), serta membungkus objek kompleks menggunakan tipe data bentukan (*struct*). Selain itu, praktikum ini menekankan penguasaan algoritma Insertion Sort dan Selection Sort untuk menyusun data baik secara membesar (*ascending*) maupun mengecil (*descending*), yang dikombinasikan dengan teknik manipulasi logika seperti pemilahan bilangan ganjil-genap dan analisis pola selisih nilai. Terakhir, materi ini memperlihatkan efisiensi algoritma pencarian biner (Binary Search) yang mensyaratkan data dalam kondisi terurut terlebih dahulu untuk mengoptimalkan waktu eksekusi program.